

Measuring the Indic Tokenizer Tax

A comparative study of Sarvam AI's tokenizers and akshara-aware subword segmentation for Indian languages

Nikhileshwar Jakka | Reg. No. [REG_NO]

B.Tech CSE (IoT & Intelligent Systems), Manipal University Jaipur

[COURSE_CODE] Natural Language Processing | Guide: [FACULTY_NAME] | November 2026

The tokenizer tax

Same FLORES sentence, translated: how many tokens compared with English?

8.29×

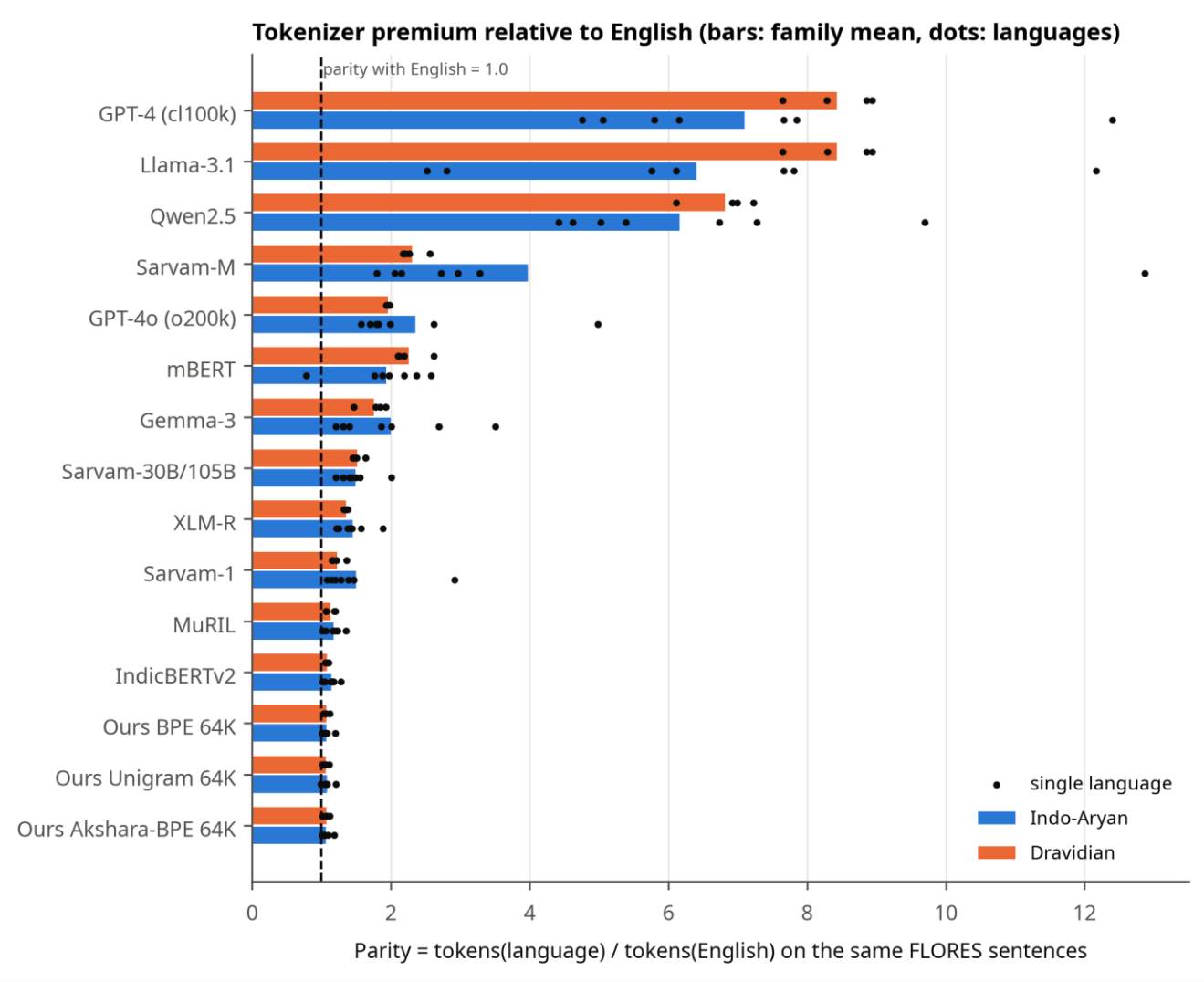
Telugu with Llama-3.1

1.93×

Telugu with GPT-4o

1.15×

Telugu with Sarvam-1



What a tokenizer does, and why aksharas matter

An akshara is the written syllable of Brahmic scripts: consonants + virama + vowel sign

Word: क्षत्रिय (kshatriya)

Akshara-respecting split (XLM-R, ours)



mBERT's actual split: virama and vowel sign orphaned



Akshara Split Rate (ASR)
% of aksharas that contain a token boundary

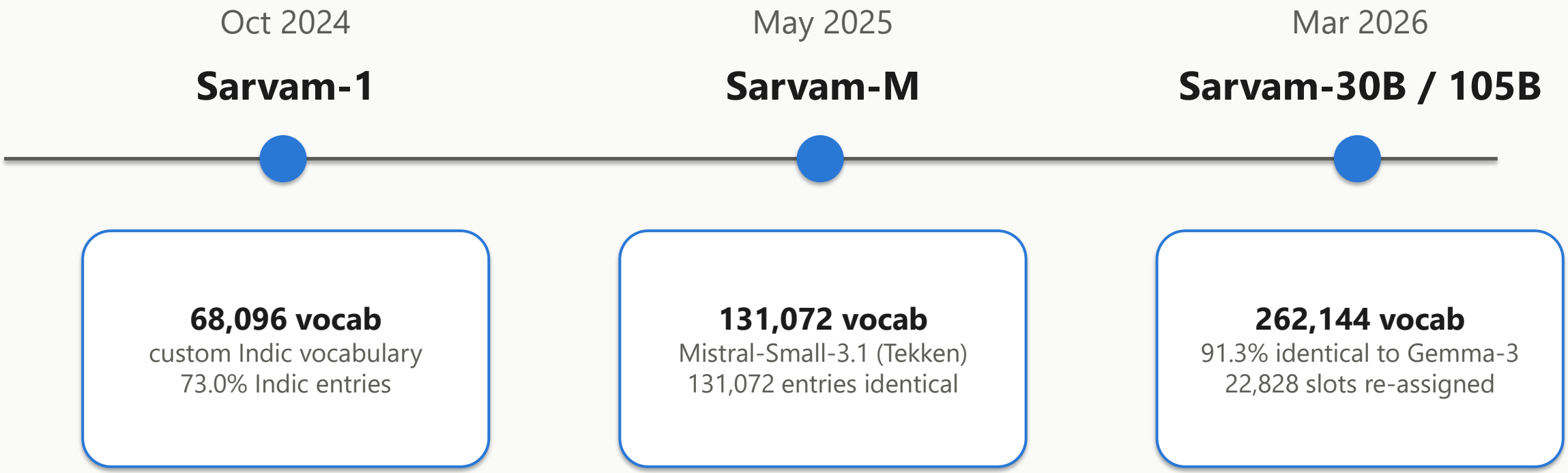
Fertility
tokens per word

Parity
tokens vs English

Exact byte spans
for every token, 4 tokenizer families

Sarvam's tokenizer journey

Three model generations, three tokenizers



Sources: Sarvam AI blog posts; vocabulary comparisons computed in this project.

Research questions

RQ1 How did Sarvam's tokenizers change, and how do they compare with global tokenizers?

RQ2 Is there a cost gap between Indo-Aryan and Dravidian languages, or between scripts?

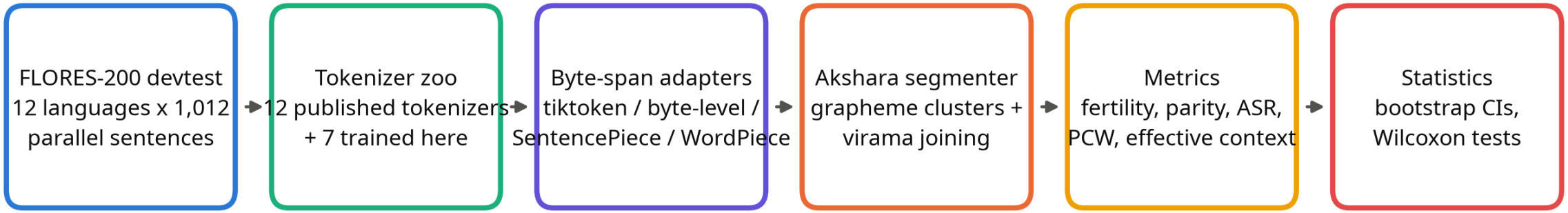
RQ3 How often do tokenizers break aksharas?

RQ4 Does akshara-atomic BPE remove broken aksharas without hurting fertility?

RQ4 is my own contribution.

Method

12 published + 7 trained tokenizers, 12 languages, 1,012 parallel sentences each



Training branch (RQ4): Sangraha 600K sentences -> akshara mapping to Private Use Area -> SentencePiece BPE -> inverse mapping

600,000

training sentences (Sangraha)

30

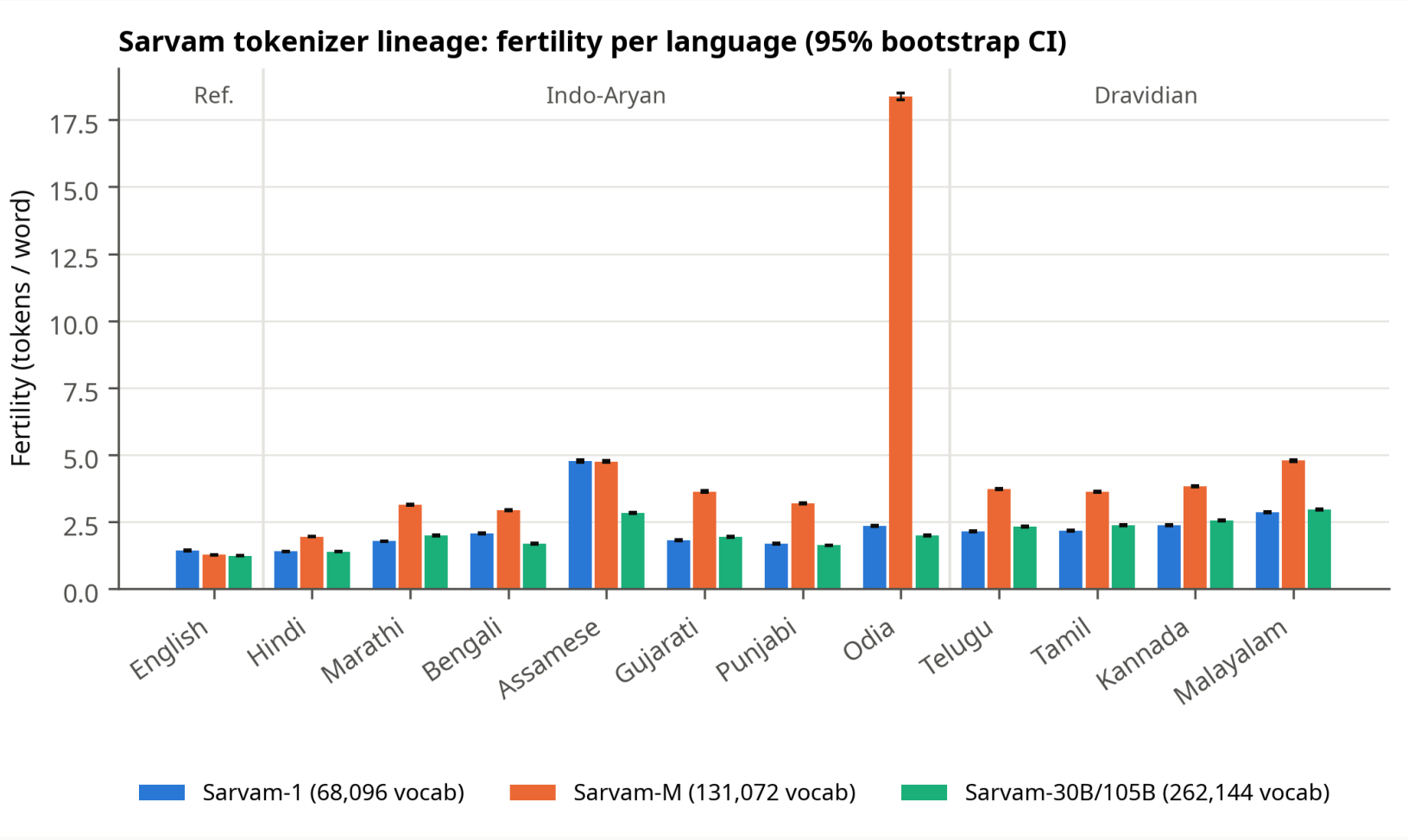
hand-checked akshara tests, 9 scripts

1,000

bootstrap resamples + Wilcoxon tests

RQ1: Sarvam-30B gained on Indo-Aryan, lost on Dravidian

Fertility per language with 95% bootstrap CIs



5

languages better with Sarvam-30B than Sarvam-1

6

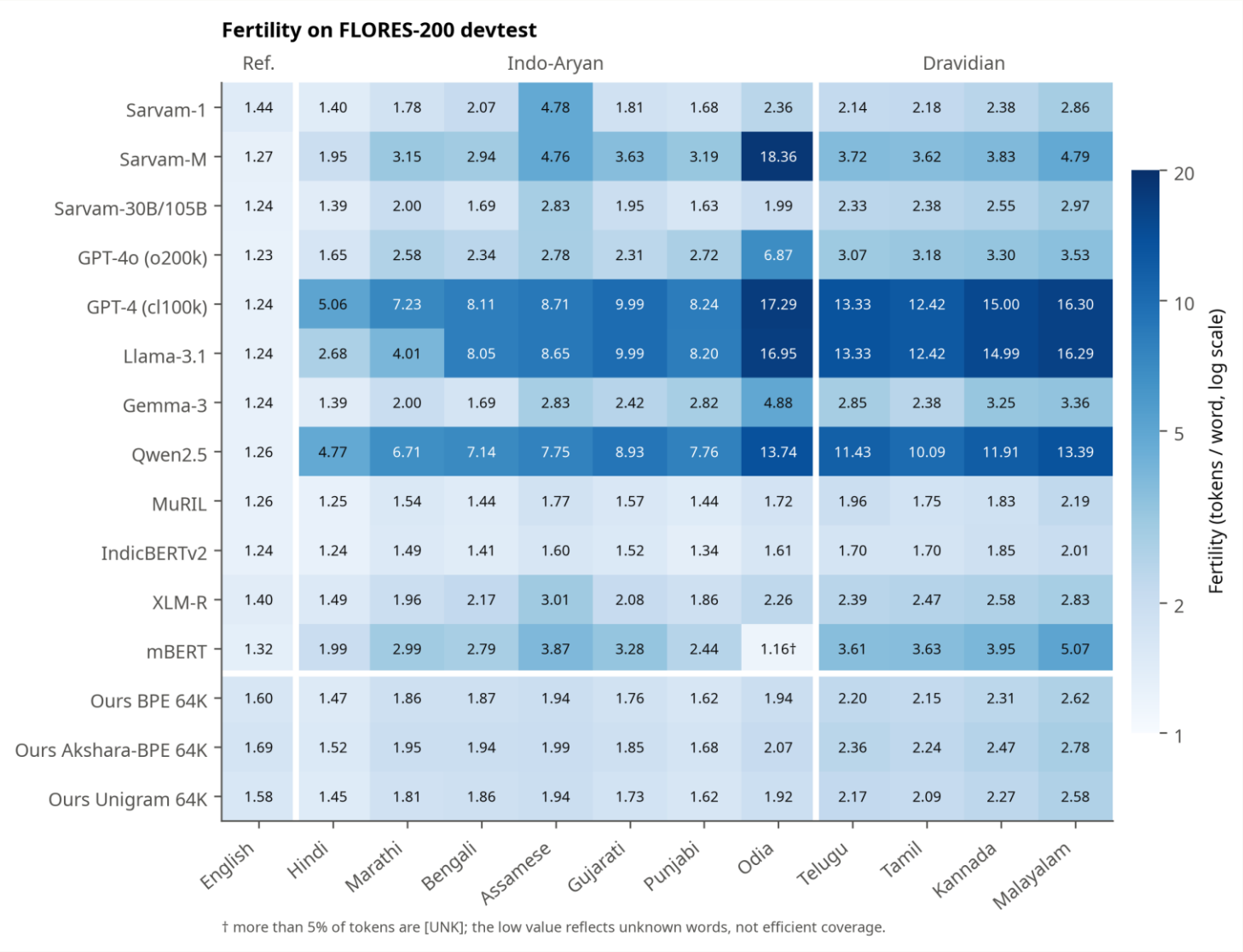
languages worse, incl. all 4 Dravidian

18.36

Sarvam-M tokens per Odia word

RQ2: Script coverage matters more than language family

Fertility across all tokenizers



1.05 → 1.32

Dravidian / Indo-Aryan fertility, Sarvam-1 → Sarvam-30B

10 / 19

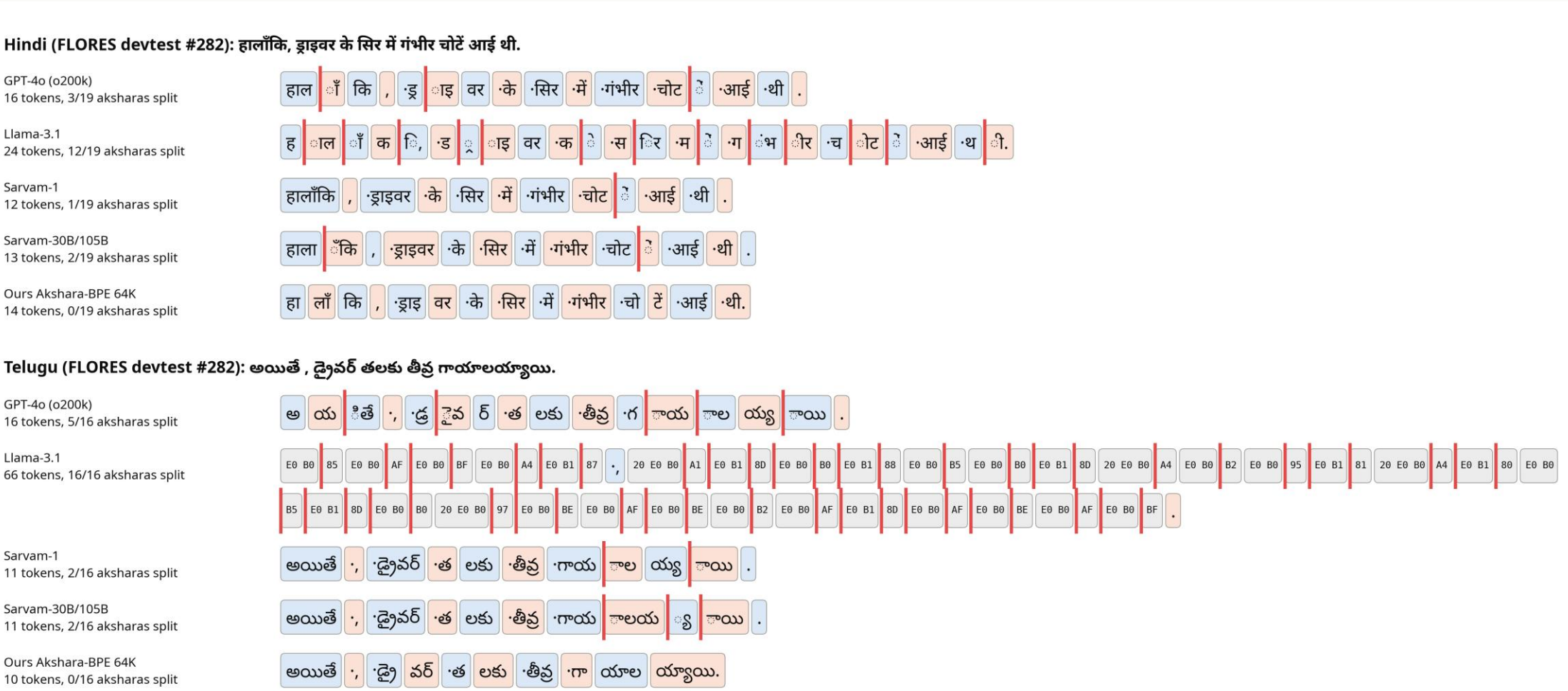
tokenizers where Dravidian parity is higher

4.98×

GPT-4o parity for Odia (worst language)

What the tokenizers actually see

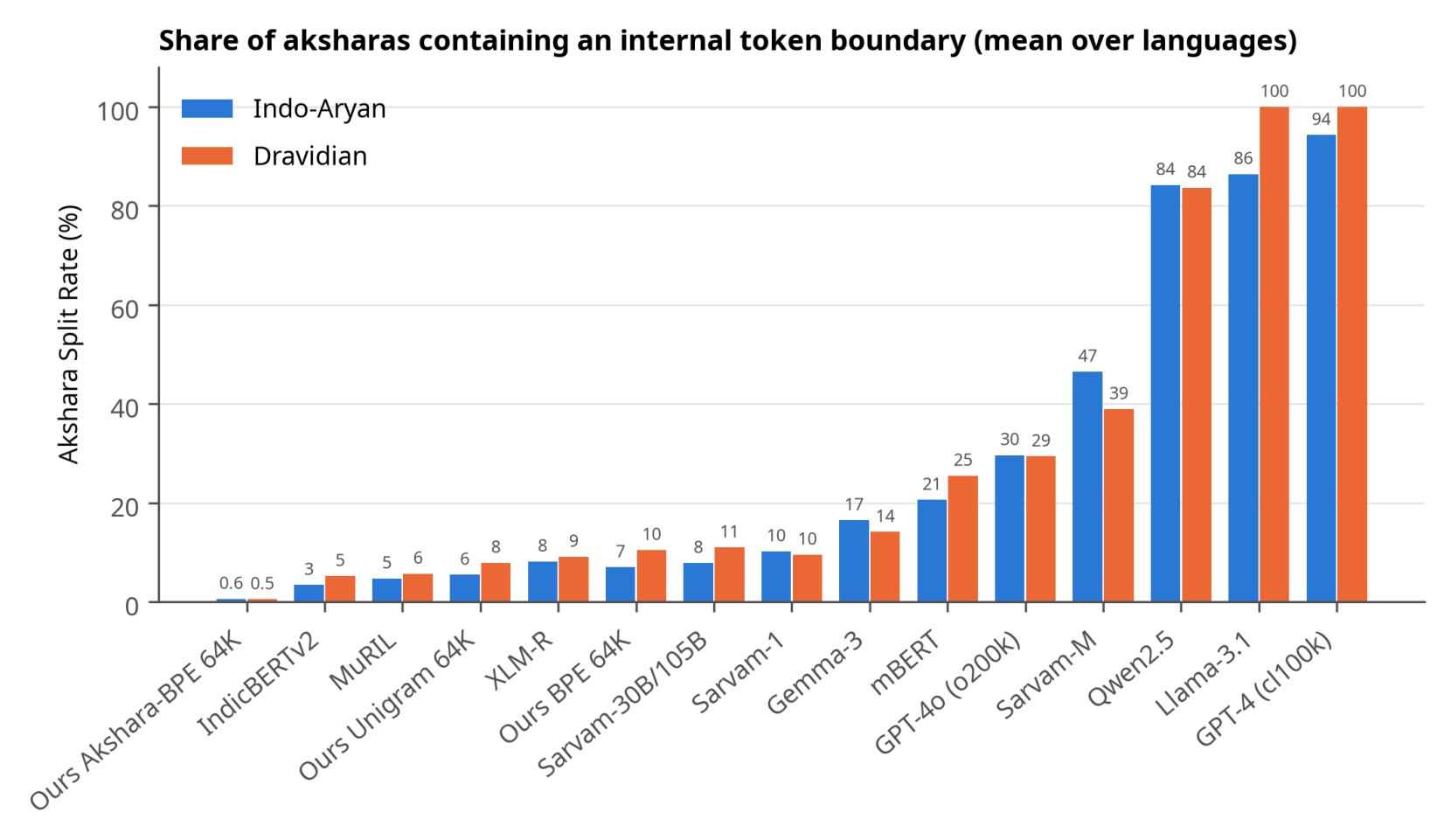
Red bars = token boundary inside an akshara; grey hex = byte fragments



Boxes = tokens (alternating colours). Grey boxes with hex = byte fragments that are not a complete character. Red bar = token boundary inside an akshara. · = space.

RQ3: Byte-level tokenizers break almost every akshara

Akshara Split Rate by tokenizer



91.4%

aksharas split by Llama-3.1

9.0%

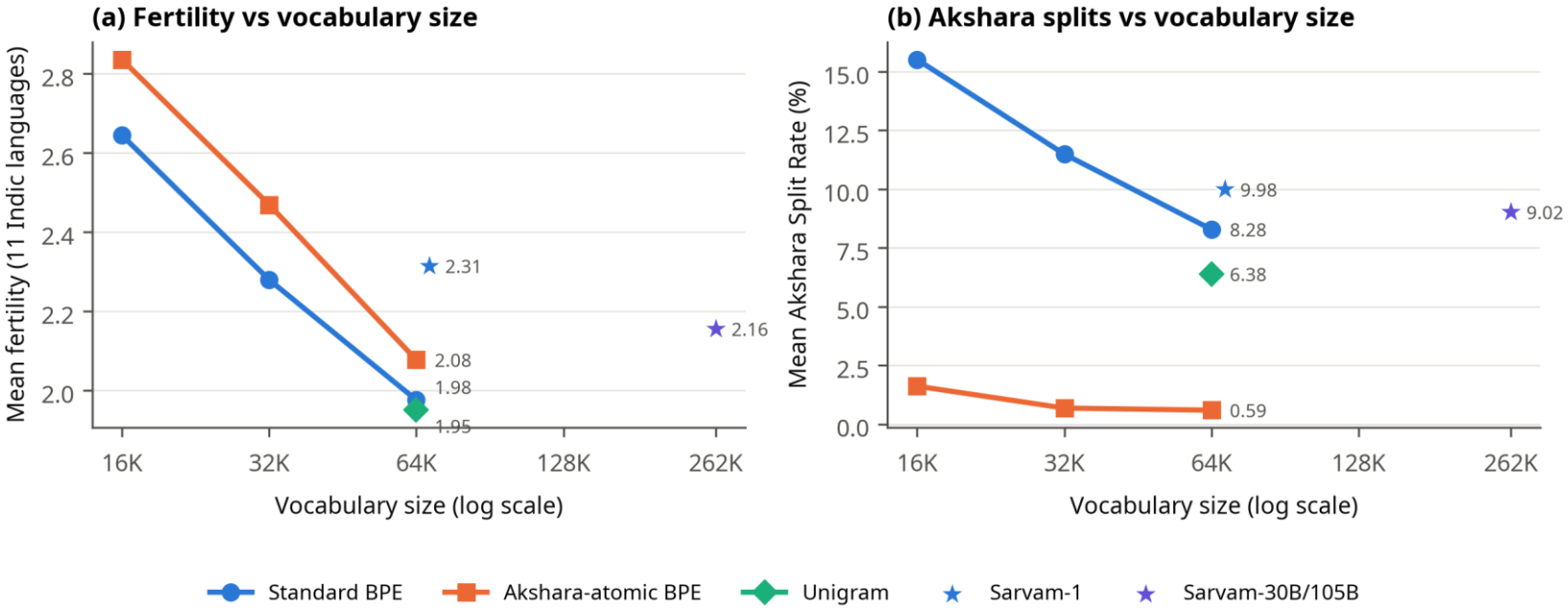
aksharas split by Sarvam-30B

1.40×

Sarvam-30B: Dravidian vs Indo-Aryan split rate

RQ4: Akshara-atomic BPE

Each frequent akshara gets one private-use symbol before BPE training



8.3% → 0.6%

Akshara Split Rate at 64K

+5.1%

fertility cost at 64K

0

mapped aksharas ever split

Every remaining split is on one of the 0.59% of aksharas outside the mapped inventory. vs Sarvam-1: fertility 2.08 vs 2.31, ASR 17× lower.

Key takeaways

91.3%

of Sarvam-30B's vocabulary is identical to Gemma-3

6 / 11

languages got more expensive from Sarvam-1 to Sarvam-30B

8.29×

Telugu token premium on Llama-3.1

−92.9%

akshara splits with akshara-atomic BPE (64K)

Limitations and future work

Limitations

- Whitespace words penalise agglutinative languages
- FLORES is a single (Wikipedia-style) domain
- Intrinsic metrics only: no models trained
- Akshara rules approximate syllables

Future work

- Train small LMs per tokenizer to test downstream effects
- Add IN22 and conversational text
- Combine akshara units with morphology-aware segmentation
- Cover all 22 scheduled languages

Thank you

Questions?

Code, data pointers, results, paper and slides:
https://github.com/jkknikhil-git/indic-tokenizer-study



Scan for the repository

Nikhileshwar Jakka | jkk.nikhil@gmail.com